Bevezetés a Rust programozásba

Gembela Gergely

6. óra: Vec<T>, Smart Pointers, Collections

https://github.com/bmeaut/VIAUBXAV090

Smart pointerek

- C++-ból ismert koncepció
- Rust-ban az ownership miatt gyakran „birtokolják” az adatot amire mutatnak
  - A referenciák csak „kölcsönöznek”
- Itt a standard könyvtár elemeit tárgyaljuk
  - Lehetséges saját implementálása is, egyéb könyvtárak is biztosítanak implementációkat

Box<T>

- Egy doboz… Ami egy általa „birtokolt” heap allokációra mutat
- Az ownership előadáson bemutatott `Drop` trait segítségével feltakarít maga után
- A modern C++ `std::unique_ptr`-éhez hasonló (de nem ABI szinten!)
- A segítségével szokás rekurzív típusok rekurzív mezőit tárolni
- Pl. a Huffman algoritmus fájában is használható volt

Rc<T>

- Egyszálas környezetben használható referenciaszámlált pointer
  - A `Weak<T>` egy olyan verziója, amely nem biztosít birtoklást a kezelt allokációra (pl. az `Rc::downgrade(...)` hívásával jön létre)
    - emiatt **nem** garantálható, hogy az `upgrade` során érvényes `Rc`-t kapunk vissza (Option az upgrade eredménye)
  - `clone` hatására egy új példányt kapunk, amely azonos erőforrásra mutat, de inkrementálja a referenciaszámlálót

Rc<T>

- A felszabadítás az utolsó erős referencia megszűnésekor történik
- A kezelt allokáció immutable!

```
let rc1 = Rc::new(42);
let rc2 = rc1.clone(); //strong count++
let w1 = Rc::downgrade(&rc1); //weak count++
```

RefCell<T>

- Az *interior mutability* minta „alapköve”
- Belső unsafe kóddal futásidőben biztosítja a memóriabiztonságot, de a működése hasonlít a `Box`-éra (ami viszont fordításidőben biztosítja)
- **Nem thread safe!**
  - A `Mutex<T>` a thread safe verzója, az `RwLock<T>` is gyakran használatos helyette

Arc<T>

- Az Rc többszálas környezetbe szánt verziója
- Mutex, RwLock, és egyéb típusok segítségével gyakran használatos az interior mutability minta megvalósításakor
- Atomi műveletekkel kezeli a referenciaszámlálót (ennek a sima Rc-hez képest van ára)

Rövid összefoglaló

- Az `Rc<T>` több fordításidőben ellenőrzött immutable referenciát engedélyez, lényegében több owner keletkezik
- A `Box<T>` fordításidőben ellenőrzött mut/immutable referenciákat is engedélyez, de egy owner van
- A `RefCell<T>` futásidőben ellenőrzi az elérést, ezért immutable `RefCell` belső értéke is változtatható, de pánikol ha hibázunk

A bemutatott konstrukciók kombinációi

- `Rc`+`RefCell` segítségével több ownere is lehet egy belül mutable értéknek
- `Arc` + `Mutex` vagy `RwLock` segítségével több szálon is biztonságosan kezelhető interior mutability implementációt kapunk

Körkörös referenciák

- `Rc` és `RefCell` használatával lehetséges memóriaszivárgást alkotni safe Rust-ban is
- A megoldás `Weak` referenciák használata

Pinning

- Ha le kell kössünk egy allokációt egy adott címre
  - A `Pin<Ptr>` típusba csomagolva érjük el
  - `Unpin` típusokra nincs hatása
  - Ha valami `Pin`-elve van (és a típus nem `Unpin`), garantáltan az adott címen marad (nem lesz mozgatva)

Tárolók - `Vec<T>`

- A C++ std::vector-ához hasonló
- Létrehozni vagy a `new()` metódussal, vagy a `vec![/*initial*/]` makróval lehetséges
- Olvasni vagy indexeléssel lehet (pánikol helytelen index esetén), vagy a `.get()` metódussal, amely `Option<&T>`-t ad vissza
- beszúrni a `.push()` metódussal lehetséges

Tárolók - `String`

- UTF-8 karakterláncok tárolására
- A C++ `std::string`-jével szemben nem használható byte bufferként!
  - Erre a célra használjunk `Vec<u8>`-at
- Kiterjeszthető (`push`, `push_str`)
- Ha egyéb kódolásra van szükségünk, `OsString` vagy `CString`

Tárolók - `HashMap<K, V>`

- Értékek kulcsok szerinti tárolására
- Minden tárolt érték és kulcs típusa azonos
- Beszúrni az `insert`-tel lehet
- SipHash-t használ

W6T1: Táblák (első verzió)

- Készíts egy egyszerű in-memory relációs táblakezelő rendszert
- A checkpointok mentén néha közösen is megbeszéljük, hogyan volt érdemes megoldani a részfeladatokat
- A leírás a GitHub repoban található

Köszönöm a figyelmet!